

CSCE 625 ARTIFICIAL INTELLIGENCE · MODULE 12 · LECTURE NOTES

Multiple Agents

Coordination, crowds, and emergence.

MODULE OUTCOMES

Explain why multi-agent problems are not just harder single-agent ones.

Implement coordinated multi-agent pathfinding.

Explain crowd simulation and its characteristic failures.

Explain emergence and how to design for it.

Explain the basic game-theoretic concepts an agent needs.

1 Why multi-agent is different

OTHER AGENTS ARE NOT OBSTACLES

Treating other agents as moving obstacles is the cheap approximation, and it fails in a specific and diagnosable way: both agents plan around the other's *predicted* path, and the act of planning invalidates both predictions.

Two agents meeting in a corridor each step aside, observe that the other has also stepped aside, and each step back — and the oscillation is not a bug in either agent. **Each is behaving correctly given a model of the other as non-reactive**, and the model is what is wrong.

RVO fixes it by assumption (Module 11 §3): assume the other agent is also avoiding, and take only half the required correction yourself. **It is a convention, shared by construction**, and it works because both agents run the same code.

And the general lesson is that coordination requires either communication or a shared convention. In a game you can simply impose the convention, because you wrote every agent — **which is a luxury robotics and economics do not have**, and it is why multi-agent game AI is substantially easier than the general problem. **Exploit it rather than solving the harder problem.**

2 Multi-agent pathfinding

THE PROBLEM: k agents with k start/goal pairs, and paths that never collide -- neither occupying the same cell at the same time, nor swapping through each other.

COUPLED -- search the JOINT state space directly.
Optimal and complete. The joint state space is the PRODUCT of the individual state spaces, so it is exponential in k . Viable to perhaps $k = 5$ on a small map, and not beyond.

DECOUPLED / PRIORITISED -- plan agents one at a time in priority order, each treating the already-planned agents as moving obstacles in space-time.

Fast, linear in k , and INCOMPLETE: it can fail on a solvable instance, because an early agent's path may make a later agent's goal unreachable. Used widely anyway, because it is fast and usually works.

CONFLICT-BASED SEARCH (CBS) -- plan every agent independently, find the first collision, and BRANCH on it: one child forbids agent A from that cell at that timestep, the other forbids agent B. Replan only the newly constrained agent, and recurse.

OPTIMAL, and far faster than coupled search in practice, because MOST AGENTS NEVER CONFLICT and only the conflicts cost anything.

CBS IS BRANCH AND BOUND (CSCE 669 Module 09) WITH COLLISIONS AS THE BRANCHING VARIABLE -- and it is the same "learn from the conflict rather than retreating from it" idea as CDCL (Module 08 section 2).

CBS is the one worth knowing. It is optimal, practical at useful scales, and **structurally identical to two other algorithms in this program** — which is worth noticing, because 'search the relaxed problem, find the violated condition, branch on it' is a **general pattern** rather than three coincidences, and recognising it lets you construct the algorithm for a new problem rather than looking one up.

3 Crowds

Model	How it works	Characteristic failure
Social forces	Agents exert repulsive forces on each other; the goal exerts attraction.	Agents visibly jitter, and they interpenetrate under pressure because a force-based method has no hard constraint.
RVO / ORCA	Collision avoidance computed in velocity space, choosing the nearest velocity that is collision-free for a time horizon.	Deadlock in dense bidirectional flow — see the note below.

Model	How it works	Characteristic failure
Continuum / flow models	Treat the crowd as a fluid with a density and velocity field.	Individuals lose agency: there are no personal goals and no individual decisions, which is fine for background and wrong for anything the player interacts with.
Flow fields	One shared direction field per destination (Module 04 §3).	Everyone takes exactly the same route, which looks mechanical and congests one corridor.
Hybrid	A flow field for global direction plus local avoidance for immediate conflicts.	What production systems use, and the tuning is empirical — there is no principled way to set the weights.

Dense bidirectional flow through a narrow gap deadlocks every purely local method. Two opposing streams meet in a doorway, each agent correctly refuses to collide, and nothing moves — and no amount of parameter tuning fixes it, because the information needed (who should go first) is not available locally. **The production fix is global:** explicit queueing, lane assignment by direction, or a capacity limit on the gap. **Knowing that this case is structurally unfixable locally prevents a great deal of wasted tuning,** which is the main reason to state it.

EMERGENCE IS DESIGNED, NOT HOPED FOR

Flocking from three local rules is the famous example (Module 11 §3) **and it is genuinely striking** — complex, coordinated, lifelike collective motion from purely local reflex behaviours with no communication and no plan.

But emergence is not free, and the famous example is selected. Most simple rule sets produce either nothing interesting or undirected chaos, and **Reynolds' three rules were found by iteration against a target, not derived from principles.**

So designing for emergence means tuning toward a target behaviour, with a way to measure whether you have achieved it — which means defining the target first, in measurable terms. 'The crowd should look alive' is not a target; 'agents should maintain 0.5 to 1.5 metres of separation while sustaining 80% of free-flow speed' is.

And emergent systems are hard to constrain. 'The crowd should never block this doorway' is not expressible as a local rule, because no individual agent can perceive the global condition — **so you end up adding global overrides, and that is where most of the engineering goes.** **Which is the honest shape of emergent design:** a local system that produces the texture, plus global machinery that enforces the requirements, and the second is larger than the first.

4 Reasoning about other agents

- **Nash equilibrium:** a joint strategy where no agent can improve its outcome by changing its own strategy alone. **It may not be unique, and it may be bad for everyone — mutual defection in the prisoner's dilemma is an equilibrium**, which is the standard illustration that equilibrium does not mean good.
- **Dominant strategy:** one that is best regardless of what others do. **When one exists the problem is easy and reasoning about others is unnecessary**; usually none exists, which is why the subject is hard.
- **Mixed strategies: randomising over actions is sometimes strictly optimal** — rock-paper-scissors has no good deterministic strategy. **Which means a predictable agent is exploitable, and game AI should sometimes be deliberately random**, not as a concession but as the correct play.
- **Cooperative games and coalitions:** which agents should team up, and how the joint gain should be divided (the Shapley value and the core) — which is CSCE 717's subject, arriving in Semester 12.
- **And mechanism design: choose the rules of the interaction so that self-interested behaviour produces the outcome you want. This is the designer's tool and it is the one to take away:** you are not a player in the game, you are the person who writes it, so you can make cooperation or competition the rational choice rather than hoping agents or players choose it.

WHAT A GAME DESIGNER SHOULD TAKE FROM THIS

Impose conventions rather than negotiating them. You control every agent's code, so coordination is a shared-state and shared-convention problem rather than a game-theoretic one (§1) — and solving the easy version deliberately is better engineering than solving the hard version by accident.

Use CBS or prioritised planning for tactical coordination (a squad moving through a building, where the agent count is small and the paths matter), **and flow fields plus local avoidance for crowds** (where the count is large and individual paths do not).

Expect the deadlock case and handle it globally. Every local method fails at a congested bidirectional gap (§3), so design the global mechanism up front rather than discovering the failure in playtest and tuning against it.

And randomise deliberately. A deterministic agent is solved by the player on the second encounter and is thereafter a puzzle rather than an opponent — **so mixed strategies are practical rather than theoretical**, and they are the difference between a fight and a sequence the player executes. **This is Module 05 §4's point about tuned difficulty, reached from the other direction:** the agent needs unpredictability for the same reason it needs legibility — both are about what the encounter is like to play, which is the actual objective.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Sharon et al. — Conflict-Based Search for Optimal Multi-Agent Pathfinding (free)	The §2 algorithm , with the branching argument and the comparison against coupled search.
van den Berg et al. — Reciprocal n-body Collision Avoidance (ORCA) (free)	The §1 and §3 method , with a free reference implementation used widely in games.
Reynolds — Flocks, Herds, and Schools (free)	The §3 emergence example , and the paper is candid about the rules being found by iteration.
Shoham & Leyton-Brown — Multiagent Systems (free PDF)	Free in full, and the reference for §4 — game theory, coalitions, and mechanism design, aimed at computer scientists.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Reproduce the corridor oscillation** with two agents treating each other as obstacles.
- 2 **Fix it with RVO** and confirm the oscillation stops.
- 3 **Implement coupled multi-agent search** and report the state count against k.
- 4 **Implement prioritised planning** and construct an instance where it fails although a solution exists.
- 5 **Implement CBS** and compare node counts against coupled search for $k = 2$ to 8.
- 6 **Simulate 200 agents with a flow field plus local avoidance** and measure throughput through a doorway.
- 7 **Create the bidirectional deadlock** and confirm no parameter setting resolves it.
- 8 **Add a global lane assignment** and report the throughput improvement.
- 9 **Implement flocking and define a measurable target**, then tune toward it and report the metric.
- 10 **Build a deterministic agent and a mixed-strategy one** and have someone play both twice. Report what they said.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why does treating other agents as obstacles fail, specifically?
- 2 What does RVO assume, and why can a game rely on it?
- 3 Compare coupled, prioritised, and conflict-based multi-agent pathfinding.
- 4 What is CBS structurally equivalent to, and in which two other courses?
- 5 Name five crowd models and the failure of each.

- 6 Why does dense bidirectional flow deadlock every local method?
- 7 Why is emergence designed rather than hoped for?
- 8 Why are emergent systems hard to constrain?
- 9 Name five game-theoretic concepts and which is the designer's tool.
- 10 Why should a game agent randomise?